



UNIVERSIDAD SAN FRANCISCO

Universidad San Francisco de Quito

Fundamentos de Ciencia de Datos: End to End

Memo técnico — PSet 2

Subvenciones con Ruta

Pipeline ELT end-to-end: OpenAlex → Snowflake con Kestra, dbt y Spark

Estudiantes: Gabriel Cázares

Mateo Ríos

Mateo Díaz

Yahashua Paredes

Semestre: Primer Semestre 2026-2027

Fecha: 4 de octubre de 2026

Resumen

Subvenciones con Ruta es la evolución de *Becas con Ruta* (PSet 1): en lugar de etiquetar a mano la pertinencia perfil-beca, usamos el historial real de financiamiento de OpenAlex como verdad de terreno. Si una publicación de un investigador fue financiada por un award, ese perfil fue pertinente para ese financiamiento. Este PSet construye la tubería que deja esos datos limpios, modelados y aplanados en Snowflake.

El pipeline corre de punta a punta con `docker compose up` y una ejecución de Kestra. En la cuenta del equipo dejó **17,1 M de awards** y **1,98 M de publicaciones** en Bronze, `dbt build` con 127/127 nodos en verde y una OBT de **1 882 074 filas**, idéntica en conteo y llaves a su hecho base. El código está en github.com/matt-rios-4/GrantMatch (carpeta `pset_2`).

1. Arquitectura

El flujo es **ELT**: los datos se cargan crudos y se transforman dentro de Snowflake (Figura 1). (1) Kestra ejecuta un `COPY INTO` desde el bucket público de OpenAlex hacia BRONZE, sin pasar los archivos por la máquina local; (2) dispara `dbt build`, que construye SILVER y GOLD y corre los tests; (3) dispara el job de Spark, que lee GOLD y escribe la OBT. Kestra invoca dbt y Spark con `docker exec` sobre los contenedores del compose, así cada herramienta corre con sus dependencias fijas.

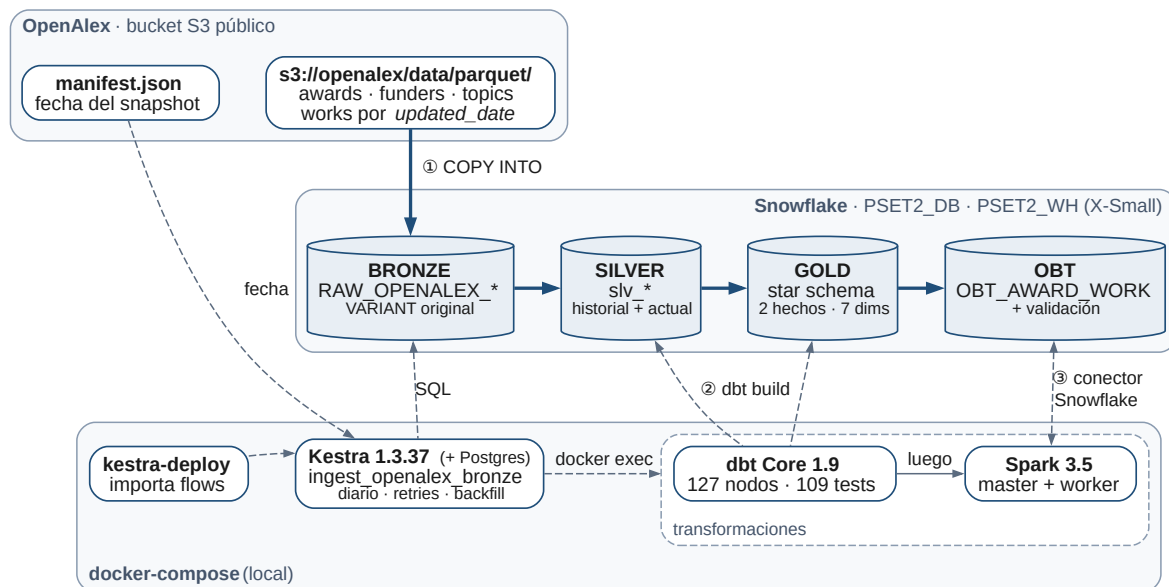


Figura 1: Arquitectura implementada. Las flechas gruesas son el flujo de datos y las punteadas, la orquestación. La infraestructura local vive en un solo `docker-compose.yml`; Snowflake es el destino y el motor de las transformaciones.

Decisiones que afectan la reproducibilidad:

- **Versiones fijas.** Kestra quedó en 1.3.37: la etiqueta `latest` ya apuntaba a 2.0.4 y cada integrante podía terminar con una versión mayor distinta. dbt 1.9 y Spark 3.5.3 también están fijados.
- **Despliegue de flows por API.** Un servicio `kestra-deploy` importa los flows en cada `docker compose up` y falla si Kestra rechaza alguno. Reemplazó la sincronización por `micronaut.io.watch`, que en Kestra 1.3 no cargaba los archivos y escribía copias en el repo.

- **COPY INTO en vez de tablas externas.** Silver aplanar y deduplicar 17 M de registros en cada corrida; leer siempre desde S3 sería más lento y caro que micro-particiones nativas.
- **Seguridad.** Kestra se publica solo en 127.0.0.1 (tiene `docker.sock` montado), las credenciales viven en `.env` (fuera de git) y Spark acepta llave RSA además de password.

2. Ingesta

Fuente. Snapshot público de OpenAlex en S3 (Parquet, sin credenciales) [1]: `awards` (17,1 M de registros, 5 GB), `funders`, `topics` y `works` (476 M de registros en 2040 archivos, 707 GB, particionado por `updated_date`). La fecha del último snapshot sale de `works/manifest.json`.

Granularidad y volumen. Los catálogos (`topics`, `funders`, `awards`) se cargan completos. `Works` se carga por partición diaria, con un tope de archivos por día: una partición completa pesa **136,6 GB** (160 archivos), inviable para una cuenta trial. Cada fila de `Bronze` es un registro de un archivo, guardado íntegro en `RAW (VARIANT)` junto con `_SOURCE_FILE`, `_SOURCE_ROW_NUMBER` y `_LOADED_AT`.

Proceso de carga (flow `ingest_openalex_bronze`). Crea de forma idempotente el stage externo y las 4 tablas `Bronze`, y corre un `COPY INTO` por catálogo [2]. Para `works` calcula un *watermark* (la última partición ya cargada, leída de `_SOURCE_FILE`) y carga las particiones entre ese punto y la fecha del snapshot, con un tope de `works_max_days` días por corrida. `ON_ERROR = ABORT_STATEMENT` revierte el `COPY` completo si un archivo falla, y los metadatos de carga de Snowflake impiden recargar un archivo ya cargado: re-ejecutar no duplica.

Frecuencia y trigger. Schedule diario a las 06:00 UTC con `recoverMissedSchedules: LAST` y `concurrency: 1`, para que una corrida manual y la programada nunca carguen en paralelo [3]. OpenAlex no publica a diario sino snapshots periódicos: los días sin snapshot nuevo la corrida termina en 15 s (“works al día”) y, cuando aparece uno, el watermark se pone al día solo, sin huecos.

Errores y retries. Todas las tareas de Snowflake heredan un retry exponencial (30 s, 60 s, 120 s...; `maxInterval` de 10 min; 5 intentos) y tienen timeout. El retry de Kestra no admite jitter (lo verificamos: rechaza el campo); lo aporta el driver JDBC de Snowflake en sus reintentos HTTP. Si una tarea agota sus intentos, un bloque `errors` registra la falla.

Backfill. Los históricos de `works` se cargan con los inputs `backfill_from / backfill_to`; un backfill no mueve el watermark hacia atrás. La evidencia de las pruebas en la cuenta del equipo (warehouse X-Small):

Prueba	Resultado
Primera carga completa	<code>topics</code> 4516 filas (3 s) · <code>funders</code> 45 661 (5 s) · <code>awards</code> 17 139 262 (1 min 34 s) · <code>works</code> 558 403 (1 min 18 s)
Re-ejecución sobre lo ya cargado	0 filas nuevas en las 4 tablas (15 s)
Modo automático sin snapshot nuevo	“works al día”: 0 filas (15 s)
Backfill 2026-09-20 a 2026-09-21	754 922 <code>works</code> nuevos (1 min 35 s); <code>dbt build incremental</code> 127/127 (2 min 34 s)
Falla real (Spark sin memoria)	tarea en <code>FAILED</code> y ejecución del bloque <code>errors</code>

3. Calidad de datos

Perfilamos Bronze antes de transformar (`dbt/analyses/data_quality_profile.sql`: 32 métricas sobre 17,1 M de awards y 1,98 M de works; la tabla completa está en el Anexo A). Revisamos **completitud** (15 métricas), **precisión** (3), **consistencia** (4), **validez** (4) y **unicidad** (6). La regla general: **ninguna corrección borra un award o un work válido por un campo malo**; se anula el campo, se marca con una bandera y la decisión queda documentada en el modelo.

Problema	Evidencia	Acción	Justificación
Awards sin descripción	70,62 % (12,1 M de 17,1 M)	Se conservan con <code>has_description = false</code>	Sin texto no se vectorizan, pero cuentan para el análisis de financiamiento
Awards sin tema (<code>primary_topic</code>)	93,04 % (15,9 M)	Van al miembro UNKNOWN de <code>dim_topic</code>	Es un hueco de OpenAlex (Silver cuadra exacto con Bronze); el filtro por dominio usa el tema del work
Awards sin monto	62,79 %	Se deja NULL, sin imputar	No falta al azar: depende del financiador (MNAR)
Montos ≤ 0	1,28 % de los awards con monto	<code>amount = NULL</code> , <code>is_amount_invalid</code>	No es un monto real; es “no informado”
Montos en otra moneda	54,88 % de los awards con monto	<code>amount_usd</code> solo si <code>currency = 'USD'</code>	Sumar EUR con USD no tiene sentido; no convertimos (Limitaciones)
Montos inverosímiles (precisión)	52 awards $\geq$ USD 1 000 M: 0,002 % de los awards, pero el 17,5 % del dinero en USD	Se conservan sin corregir	No hay cómo verificar el monto real; queda como limitación abierta
Fin antes del inicio	2,59 % de los awards con ambas fechas	<code>end_date = NULL</code> , <code>has_inconsistent_dates</code>	Viola el dominio; la duración sería negativa
Inicio solo con año (precisión)	4,42 % de los awards con inicio (317 566)	1 de enero + <code>is_start_date_from_year</code>	Conserva la dimensión de tiempo; la bandera permite excluir duraciones aproximadas
Works fechados el 1 de enero (precisión)	24,30 % (esperado por azar: 0,27 %)	Se conserva la fecha	En esos works solo vale el año: el análisis temporal usa <code>publication_year</code>
Works sin abstract	27,88 % (552 729 de 1,98 M)	Se conservan con <code>has_abstract = false</code>	Siguen siendo publicaciones financiadas; queda el título
Autorías sin <code>author_id</code>	11,19 % (845 078)	Se descartan en el puente	Sin llave no hay perfil de investigador
Autor repetido en un work	0,46 % (31 069)	Se deja una vez	Evita contar doble una autoría
Versiones y recargas	0 copias exactas en esta carga	Historial <i>append</i> + estado actual con la versión más reciente	OpenAlex reemite registros entre snapshots; sin esto se duplicarían montos

Además, los IDs llegan como URL (<https://openalex.org/G...>) y se normalizan al ID corto para que la misma entidad tenga la misma llave en todas las tablas, y se limpian etiquetas HTML en el 0,81 % de las descripciones (frecuentes en NSF), porque ensucian los embeddings.

4. Transformaciones y modelado

Bronze (Kestra) guarda el registro original en una columna `VARIANT`: conserva el dato tal cual, tolera los campos nuevos que OpenAlex agrega y deja intactas las listas anidadas. **Silver** (dbt) tipa, limpia y normaliza. Los historiales (`slv*_history`, grain registro + versión) son incrementales *append*, como pide la regla del equipo para eventos inmutables. El estado actual (`slv_awards`, `slv_works`) es incremental *merge* [4]: es una excepción deliberada, porque con

append un award actualizado aparecería dos veces; el merge solo toca la última carga y nunca pisa una versión nueva con una vieja. Las listas aplanadas (autorías, enlaces work–award) usan *delete+insert* por work. **Gold** (dbt) es el star schema de la Figura 2 [5] y sigue el mismo criterio: **fact_awards** y las dimensiones de award, work, author e institution son incrementales *merge* por su llave; **fact_award_works** y el puente, *delete+insert* por work; y los catálogos chicos (funder, topic, date) se reconstruyen como tabla.

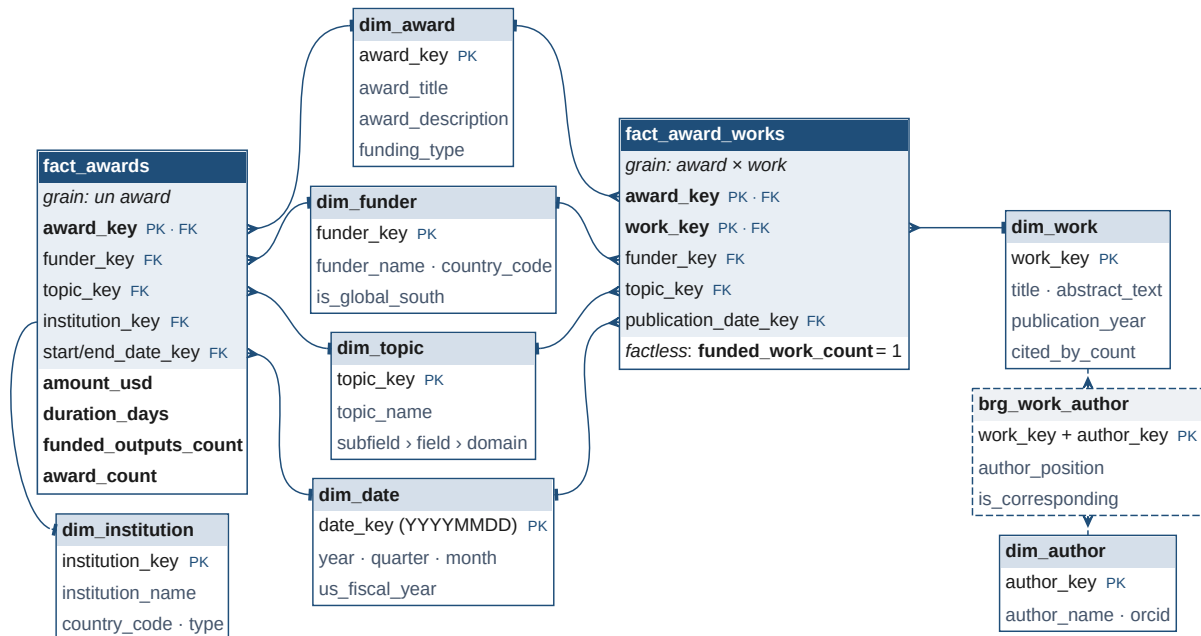


Figura 2: Modelo dimensional de la capa **GOLD**: dos hechos que comparten las dimensiones conformadas del centro. Todas las FK son NOT NULL y tienen test **relationships**; lo que no se puede resolver apunta al miembro UNKNOWN (o -1 en fechas).

- **fact_awards**. Grain: un award otorgado (17 139 262 filas). Métricas aditivas: **amount_usd**, **duration_days**, **funded_outputs_count** y **award_count**.
- **fact_award_works**. Grain: una publicación financiada por un award (award × work, 1 882 074 filas). Es un hecho sin métricas y la verdad de terreno del recomendador; no lleva el monto, que se repetiría por publicación e inflaría el dinero.
- **Dimensiones**: award (texto a vectorizar), funder, topic (jerarquía desnormalizada), institution, date (YYYYMMDD y año fiscal de EE. UU.), work y author. El puente **brg_work_author** resuelve la relación muchos a muchos entre works y autores.

Tests. 109 tests de dbt (127 nodos con los modelos), todos en verde: **not_null** y **unique** en cada llave, **relationships** en todas las FK de Gold y reglas de negocio reales (monto > 0, **amount_usd** solo en USD, fin ≥ inicio, año posible, posición de autor válida, y un test que verifica que **fact_awards** tiene exactamente una fila por award de Silver). Las dependencias usan **source()** hacia Bronze y **ref()** en todo lo demás.

5. Spark y OBT

El job `spark/jobs/build_obt.py` lee Gold con el conector Spark–Snowflake, construye **OBT.OBT_AWARD_WORK** (62 columnas) y la vuelve a escribir en Snowflake. **Grain: una publicación financiada por un award** (award_key + work_key), el mismo de **fact_award_works**. Cada fila es un ejemplo positivo (**label** = 1) con el texto del award y el de la publicación

(`award_text`, `work_text`, listos para `CORTEX.EMBED_TEXT_768`) y el contexto de ambos lados: funder, institución, monto, tema, año, citas y autores.

Construcción. Todos los joins son muchos a uno desde el hecho base; `dim_topic` y `dim_date` entran dos veces (*role-playing*, con prefijos `award_` y `work_`). Los autores, la única relación muchos a muchos (6,0 por work financiado con autores; mediana 4, máximo 4 902), se agregan primero a una fila por work (lista de autores, primer autor, países) y recién después se unen. El job filtra dentro de Snowflake lo que la OBT usa (p. ej., 863 966 de los 17,1 M de awards).

Validación de los joins. El job aborta sin escribir si falla cualquiera de estos chequeos, y cada corrida queda registrada en `OBT_AWARD_WORK_VALIDATION`:

Chequeo	Resultado
Cada tabla que se une es única en su llave (la causa raíz de duplicados)	12 de 12 tablas únicas
Ningún join queda sin pareja (un marcador por join, nulo solo si no hubo match)	0 huérfanos en 10 joins
<code>COUNT(OBT) = COUNT(fact_award_works)</code> y (<code>award_key</code> , <code>work_key</code>) único	1 882 074 = 1 882 074, pares distintos
Control: unir el puente de autores sin agregar	daría 23 607 880 filas (12,5×)

Además, 19 pruebas locales con datos sintéticos verifican que el job *sí aborta* ante una dimensión duplicada, una FK huérfana o un hecho base duplicado. La OBT se reescribe completa (`overwrite` sobre tabla temporal y swap), así que re-ejecutar da el mismo resultado; con 1,88 M de filas tarda 19 min en el cluster local (el executor necesitó 2 GB: con 1 GB caía por falta de memoria).

Star schema u OBT. El star schema es para análisis y BI (cuánto financia cada funder por área y año): cada métrica vive una vez en su grain. La OBT es para entrenar y servir el modelo: una tabla ancha, sin joins al momento de la inferencia. No sirve para sumar dinero, porque el monto de un award se repite en cada publicación que financió.

6. Batch vs. streaming

La fuente ya es batch. OpenAlex no emite eventos: publica snapshots periódicos en S3 (al 4-oct-2026, el último era del 23-sep) y un solo día de works pesa 136,6 GB. Ninguna arquitectura entrega datos más frescos que el snapshot, y procesar esos volúmenes evento por evento costaría más que un `COPY INTO` por partición. **La latencia del negocio se mide en semanas:** las convocatorias abren con meses de anticipación, el perfil de un investigador cambia cuando publica y una oficina de transferencia revisa recomendaciones semanal o mensualmente. **El costo favorece lotes:** los embeddings de Cortex se cobran por token y la similitud es un cruce autores × awards; en lotes se vectoriza solo lo nuevo y se recalculan los rankings una vez, con un warehouse que se suspende a los 60 s.

Qué justificaría streaming: (1) una fuente de eventos, como un feed de convocatorias en tiempo real (webhooks o RSS de grants.gov o de los funders); (2) decisiones con ventana de horas, como fondos de emergencia que cierran en días; o (3) personalización en línea, con investigadores que editan su perfil y esperan recomendaciones al instante. Entonces cambiaríamos la ingesta a una cola (Kafka o Kinesis) con Snowpipe Streaming, las transformaciones a Dynamic Tables o micro-batches, y calcularíamos embeddings al llegar cada convocatoria; el histórico de OpenAlex seguiría por lotes.

7. Limitaciones

- **Works es una muestra:** 1–2 de hasta 160 archivos por partición y 4 particiones de septiembre (1,98 M de 476 M works, 0,4 %); la cobertura de enlaces award–work crece con más particiones.
- **Texto del award escaso:** el 89,2 % de los pares de la OBT no tiene ni título ni descripción del award; el perfil de cada award deberá construirse con los abstracts de las publicaciones que financió (79,2 % de los pares los tiene).
- **Sin casos negativos observables:** no sabemos si alguien postuló y fue rechazado o nunca postuló.
- **Montos:** no se convierten monedas (el análisis de dinero usa solo USD, 4,2 % de los pares) y los 52 awards inverosímiles (máximo USD 373 109 M) siguen en Gold: toda suma por funder o área debe excluirlas.
- **Borrados de OpenAlex:** `deleted_ids.csv.gz` no se procesa; works y awards eliminados o fusionados permanecen en Silver y Gold.
- **Recarga de catálogos:** cada snapshot reescribe sus archivos y se vuelven a cargar (Silver descarta las copias); se paga ese cómputo una vez por snapshot.
- **Enlaces tardíos:** si un work llega antes que el award que lo financió, el enlace entra a Gold solo con un `--full-refresh` de `fact_award_works`.
- **Operación local:** el trigger debe quedar activo en una sola instancia de Kestra del equipo, y el stack completo necesita unos 8 GB de RAM en Docker.

Referencias

- [1] OpenAlex, *Snapshot data format y Awards*, 2026. [En línea]. Disponible: <https://docs.openalex.org>
- [2] Snowflake, *COPY INTO <table> y Loading using pattern matching*, 2026. [En línea]. Disponible: <https://docs.snowflake.com>
- [3] Kestra, *Retries, Schedule trigger y Backfill*, 2026. [En línea]. Disponible: <https://kestra.io/docs>
- [4] dbt Labs, *Incremental models*, 2026. [En línea]. Disponible: <https://docs.getdbt.com>
- [5] R. Kimball y M. Ross, *The Data Warehouse Toolkit*, 3.^a ed. Indianápolis, IN: Wiley, 2013.

A. Anexo: perfil de calidad completo

Bronze en la cuenta del equipo, 4 de octubre de 2026.

ID	Entidad	Dimensión	Problema	Afectados	Total	%
A01	awards	Compleitud	Sin <code>award_id</code>	0	17 139 262	0,00
A02	awards	Unicidad	Copias exactas por recarga	0	17 139 262	0,00
A03	awards	Unicidad	Más de una versión	0	17 139 262	0,00
A04	awards	Compleitud	Sin descripción	12 103 179	17 139 262	70,62
A05	awards	Compleitud	Sin monto	10 761 110	17 139 262	62,79
A06	awards	Validez	Monto ≤ 0	81 720	6 378 152	1,28
A07	awards	Consistencia	Moneda distinta de USD	3 455 575	6 296 432	54,88
A08	awards	Validez	Fin anterior al inicio	165 556	6 382 858	2,59
A09	awards	Compleitud	Sin fecha ni año de inicio	9 955 757	17 139 262	58,09
A10	awards	Consistencia	Descripción con HTML	40 541	5 036 083	0,81
A11	awards	Compleitud	Sin funder	15 548	17 139 262	0,09
A12	awards	Consistencia	Funder fuera del catálogo	0	17 123 714	0,00
A13	awards	Compleitud	Sin publicaciones enlazadas	7 432 285	17 139 262	43,36
A14	awards	Compleitud	Sin <code>primary_topic</code>	15 946 100	17 139 262	93,04
A15	awards	Precisión	Fecha de inicio solo con año	317 566	7 183 505	4,42
A16	awards	Precisión	Monto $\geq$ USD 1 000 M (inverosímil)	52	2 840 857	0,002
F01–F03	funders	Compl./Unic.	Sin ID, repetidos o sin nombre	0	45 661	0,00
T01–T02	topics	Compl./Unic.	Sin ID o repetidos	0	4 516	0,00
W01	works	Compleitud	Sin <code>work_id</code>	0	1 982 427	0,00
W02	works	Unicidad	Copias exactas por recarga	0	1 982 427	0,00
W03	works	Compleitud	Sin título	1 739	1 982 427	0,09
W04	works	Compleitud	Sin abstract	552 729	1 982 427	27,88
W05	works	Validez	Año de publicación fuera de rango	13	1 967 107	0,00
W06	works	Compleitud	Mención de award sin ID	0	1 882 074	0,00
W07	works	Compleitud	Autoría sin <code>author_id</code>	845 078	7 549 000	11,19
W08	works	Unicidad	Autor repetido en un work	31 069	6 703 922	0,46
W09	works	Consistencia	Enlace a un award no cargado	0	1 882 074	0,00
W10	works	Validez	Publicaciones retractadas	959	1 982 427	0,05
W11	works	Precisión	Fechado el 1 de enero (solo el año)	478 059	1 967 107	24,30

Los 52 awards de A16 suman USD 456 739 M de un total de USD 2 610 111 M en awards en USD (17,5 %); el mayor declara USD 373 109 M. En W11, si las fechas fueran exactas, el 1 de enero tendría cerca de 1/365 de los works (0,27 %).

B. Anexo: volumen por capa

Cuenta del equipo, 4 de octubre de 2026.

Capa	Tablas	Filas
Bronze	<code>raw_openalex_awards</code> / <code>funders</code> / <code>topics</code> / <code>works</code>	17 139 262 / 45 661 / 4 516 / 1 982 427
Silver	<code>slv_awards</code> / <code>slv_works</code>	17 139 262 / 1 982 427
Silver	<code>slv_work_authorships</code> / <code>slv_work_awards</code>	6 672 853 / 1 882 074
Gold	<code>fact_awards</code> / <code>fact_award_works</code>	17 139 262 / 1 882 074
Gold	<code>dim_work</code> / <code>dim_author</code> / <code>brg_work_author</code>	1 982 427 / 4 175 242 / 6 672 853
Gold	<code>dim_funder</code> / <code>dim_institution</code> / <code>dim_topic</code> / <code>dim_date</code>	45 662 / 27 319 / 4 517 / 31 412
OBT	<code>obt_award_work</code> (62 columnas)	1 882 074

Particiones de works cargadas: 2026-09-19 (2 archivos, 669 102 filas), 2026-09-20 (1; 400 000), 2026-09-21 (1; 354 922) y 2026-09-23 (2; 558 403).

C. Anexo: reproducción

1. `cp .env.example .env` y completar Snowflake (SNOWFLAKE_ACCOUNT con región, p.ej., <locator>.us-east-2.aws), KESTRA_USER y KESTRA_PASSWORD. Guardar con fines de línea LF.
2. Ejecutar `docs/snowflake_setup.sql` una vez, como ACCOUNTADMIN.
3. `docker compose up -d`: levanta Postgres, Kestra, dbt y Spark, y `kestra-deploy` despliega los flows.
4. En <http://localhost:8080>: `pset2.ingest_openalex_bronze` → *Execute* (o esperar el trigger diario). El flow carga Bronze, corre dbt build y construye la OBT.
5. Verificación: `OBT.OBT_AWARD_WORK_VALIDATION` con `status = 'OK'` y `obt_rows = base_rows`.